

CS 233 OBJECT ORIENTED PROGRAMMING AND DESIGN PATTERNS (OOPDP)



Shamama Anwar
Assistant Professor
Department of Computer Science and Engineering
Birla Institute of Technology, Mesra.
shamama@bitmesra.ac.in
9431103880

Course Objectives

Upon completing the course, you will be able to:

- Identify the difference between procedural and OO programming.
- Construct programs using various OOP principles.
- Design UI using JAVA GUI components.
- Operate on files and strings in real life scenarios.
- Analyze thread performance and inter thread communication issues

By: Shamama Anwar

Syllabus

Module 1: Introduction to Classes, Objects and Java

- Introduction to Object Technology, Java, Understanding the Java development environment, Programming in Java, Memory concepts, Doing basic Arithmetic, Comparing entities, Classes, Objects, Methods, Strings, Primitive vs reference types.

Module 2: Control Statements, Methods and Arrays

- Basic selection statements, Iterative constructs, Relative and Logical operators, break, continue, Methods, static methods, parameter passing, argument promotion and casting, scopes, method overloading. Arrays and ArrayList in Java, Enhanced for statement, Passing arrays to methods, Multidimensional arrays, Using command line arguments.

Module 3: Object Oriented Concepts: Polymorphism & Inheritance

- Controlling access to class members, the use of this keyword, getters and setters, Composition, enum, the use of static and final, Garbage collection. Superclass and subclass, protected members, constructors in subclass, the Object class, Introduction to polymorphism, Abstract classes and methods, Assignment between subclass and superclass variables, Creating and using interfaces.

By: Shamama Anwar

Syllabus

Module 4: Exception Handling & GUI Design

- When to use exception handling, Java exception hierarchy, finally block, Stack unwinding, Chained exceptions, Declaring new exception types, Assertions, try with resources. Simple I/O with GUI, Basic GUI Components, GUI Event handling, Adapter classes, Layout managers, using Panels

Module 5: Strings, characters & Files

- Working with the String and StringBuilder class, Character class, Tokenizing strings, Regular Expressions, Files and Streams, Using NIO classes, Sequential file handling, Object serialization, JFileChooser, Introduction to threading, Introduction to Generics and lambda expressions.

By: Shamama Anwar

Books

Text Book :

- Deitel P., Deitel H., Java How to Program, 10th Edition, Pearson Publications, 2016.

Reference Book:

- Wu C. T., Object Oriented Programming in Java, 5th Edition, McGrawHill Publications, 2010.

By: Shamama Anwar

Module - I

Introduction to Classes, Objects and Java

- Introduction to Object Technology
- Java
 - Understanding the Java development environment
 - Programming in Java
 - Memory concepts
 - Doing basic Arithmetic
 - Comparing entities
 - Classes, Objects, Methods, Strings, Primitive vs reference types.

By: Shamama Anwar

Object Oriented Programming

- Types of Programming ?????
- OOP is a programming paradigm which deals with the concepts of object to build programs and software applications.
- Every **object** has a well defined identity, attributes and behavior. Objects are instances of a class.
- **Classes** from a template for an object.



By: Shamama Anwar

Characteristics of OOP

- **Classes**: A class is defined as a blueprint of an object. It is like a template. A class contains data (attributes or features) and methods (functions).
- **Objects**: Objects are instances of a class. Objects of a class will have the same attribute (but not values) and methods that are defined in the class.
- **Abstraction**: Hiding the unnecessary details from the user.
- **Inheritance**: It is the ability of one class (sub class) to inherit features from another class (super class). It aids reusability.
- **Encapsulation**: The process of binding the data and the associated methods into one unit (called the class) and hide them from the outside world (data hiding).
- **Polymorphism**: It is the ability of a method to take more than one form. It is of two types:
 - Compile time (overloading)
 - Dynamic / Run time Polymorphism

By: Shamama Anwar

Difference between OO and Procedural Programming

Procedural Language	OOP
Separate data from functions that operate on them	Encapsulate data and methods in a class
Not suitable for defining abstract types	Suitable for defining abstract types
Difficult to implement changes	Easier to implement changes
Not suitable for larger programs and applications	Suitable for larger programs and applications
Less flexible	More flexible
Less reusable	More reusable
Procedure based	Object based
Only a function calls another	Functions and Objects communicate
Eg: C, Basic, FORTRAN	Eg: JAVA, C++, VB.NET

By: Shamama Anwar

Introduction to Java

- Java is a programming language and computing platform first released by Sun Microsystems in 1995.
- The Java Runtime Environment (JRE) is what you get when you download Java software. The JRE consists of the Java Virtual Machine (JVM), Java platform core classes, and supporting Java platform libraries.

By: Shamama Anwar

Java Features

- **Simple**: Easy to learn, Similar to C / C++, does not support pointers, Memory is automatically allocated and deallocation is done by the garbage collector and must not be explicitly programmed.
- **Object-Oriented**: Supports the concept of classes and objects, Encapsulation, Inheritance, Abstraction, Polymorphism.
- **Distributed**: Java has been designed to support applications on networks. It supports different levels of connectivity through classes which also allows a Java application to open and access remote objects on the internet.
- **Interpreted**: The Java compiler generates **byte-code** instead of machine-dependent code. To run a program one has to load it into the **Java Virtual Machine**. This machine has been implemented for the most popular operating systems.
- **Robust**: Extensive compile-time checking for errors, exception handling feature

By: Shamama Anwar

11

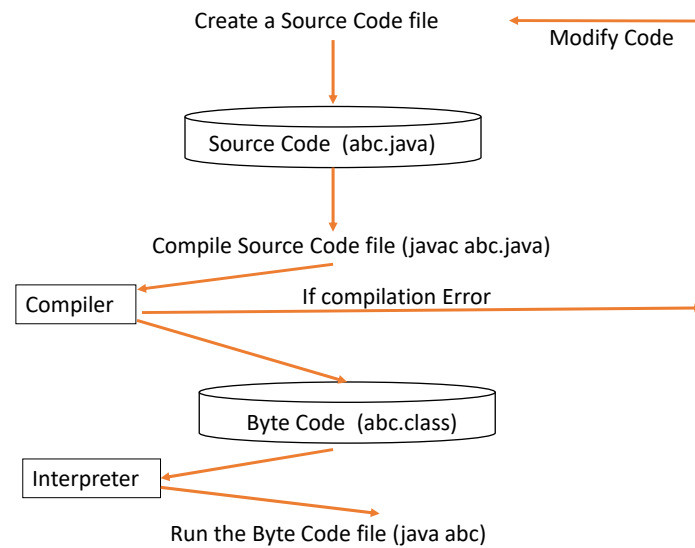
Java Features

- **Secure**: Java compiler does not handle memory layout decisions and one cannot write dynamic code, exception handling
- **Architecture-Neutral**: As Java programs are compiled to byte-code compiled programs will run on every architecture which implements the Java virtual machine.
- **Portable**
- **High performance**
- **Multithreaded**: Java has built-in constructs for multithreading. It makes programming with threads much easier by providing built-in language support for threads.
- **Dynamic**: Java manipulates memory in a dynamic way. Classes are loaded by demand even across a network. This features make it possible for a Java interpreter to download and run code from across a network.

By: Shamama Anwar

12

Creating and compiling programs

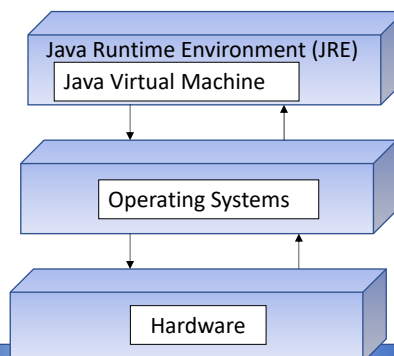


By: Shamama Anwar

11

Java Virtual Machine

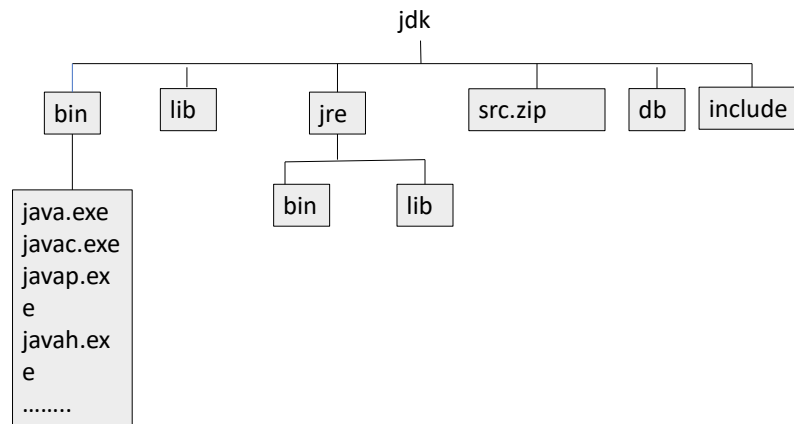
- An abstract computing machine that executes bytecode programs
- It translates byte codes into actions or operating system calls.
- It is responsible for catering to the differences between different platforms and architectures.
- It consists of:
 - Interpreter
 - Just-in-time compilation



By: Shamama Anwar

12

Java Development Toolkit (JDK)



By: Shamama Anwar

15

Classpath / Environment variables

- It is not possible to run a Java program without modifying system environment variables
- The PATH environment variable needs to be set to enable the program to run from any directory.
- The Classpath tells the JVM and other Java Applications where to find the class libraries and user defined packages.

By: Shamama Anwar

16

Programming in Java

Java Program Structure

```
// comments about the class
package details
public class classname
{
    }
    }
```

class header

class body

By: Shamama Anwar

17

A Sample Program

```
// My first program
public class MyProgram
{
    public static void main (String[] args)
    {
        System.out.println ("Hello !!!");
    }
}
```

Class

method body

By: Shamama Anwar

18

Comments

- Comments in a program are called *inline documentation*
- They should be included to explain the purpose of the program and describe processing steps
- They do not affect how a program works
- Java comments can be written as:

```
// this comment runs to the end of the line

/* this comment runs to the terminating
   symbol, even across line breaks */
```

Comments can be placed almost anywhere

By: Shamama Anwar

19

Java Programming Constructs

- Identifiers
- Keywords
- Primitive Data Types:

Datatype	Size (in bytes)
byte	1
short	2
int	4
long	8
float	4
double	8
char	2

- Variables
- Literals

By: Shamama Anwar

20

Identifiers

- *Identifiers* are the words a programmer uses in a program
- An identifier can be made up of letters, digits, the underscore character (`_`), and the dollar sign
- Identifiers cannot begin with a digit
- Java is *case sensitive* - `Total`, `total`, and `TOTAL` are different identifiers
- By convention, programmers use different case styles for different types of identifiers, such as
 - *title case* for class names - `Lincoln`
 - *upper case* for constants - `MAXIMUM`

By: Shamama Anwar

Keywords

- Java keywords are also known as reserved words.
- These give specific instructions.
- They cannot be used as literals in a program.

Variables

- Variables are declared to hold or represent values.
- Datatype needs to be specified when declaring variables.

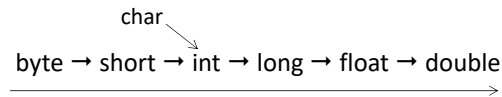
Memory Concepts

- Variable names actually correspond to locations in the computer's memory. Every variable has a name, a type, a size (in bytes) and a value.
- Memory read and write operations.

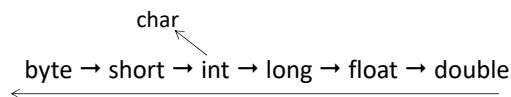
By: Shamama Anwar

Primitive Type Conversion and Casting

- Type Conversion / Widening Conversions : Automatically done in an expression from a lower data type to a higher data type.



- Casting / Narrowing Conversions : Automatically done in an expression from a lower data type to a higher data type.



By: Shamama Anwar

39

Primitive Type Conversion and Casting

```

byte b = 10;
int i = b; //implicit widening byte to int

int i = 10;
double d = 20;
d = i + d; //implicit widening int to double

byte b = 10;
short s = 30;
short z = b*s; //invalid
int i = b*s;  //valid

int i = (8.0 / 3.0); //invalid
int i = (int)(8.0 / 3.0); //Casting (Narrowing Conversion)

/*In Casting the lower datatype will contain the lower
order bits of the higher data type */
  
```

By: Shamama Anwar

40

Example

Class CastingExample

```
{
    public static void main(String args[])
    {
        int i = (int) (8.0 / 3.0);
        int j = (int) 2147483.0f;
        System.out.println("i = "+i+"j = "+j);
        byte b = (byte) 257;
        short s = (short) 65537;
        System.out.println("b = "+b+"s = "+s);
        System.out.println("Converting int to char "+(char) 67);
        double d = i * b * s * 2.0;
        System.out.println("Conversion to double:"+d);
        i = b<<2;
        System.out.println("i = "+i);
    }
}
```

i = 2 j = 2147483
b = 1 s = 1
Converting int to char A
Conversion to double : 4.0
i = 4

By: Shamama Anwar

35

User Input

1. Command line argument (Eg: 1)

```
class demo {
    public static void main(String args[])
    { System.out.println("Argument:"+args[0]); } }
```

Runtime (Command Prompt)

```
> javac demo.java
> java demo Hello
```

By: Shamama Anwar

36

User Input

1. Command line argument (Eg: 2)

`/* Sum of all numbers entered through command line argument */`

```
class demo {
public static void main(String args[])
{
    int i, s=0;
    for(i=0;i<args.length;i++)
    {
        s = s + Integer.parseInt(args[i]);
    }
}
}
```

Runtime (Command Prompt)

> javac demo.java

> java demo 1 3 5 6 7

By: Shamama Anwar

37

User Input

2. Scanner class (java.util.Scanner)

The **Java Scanner** class breaks the input into tokens using a delimiter that is whitespace by default.

Method	Description
public String next()	it returns the next token from the scanner.
public String nextLine()	it moves the scanner position to the next line and returns the value as a string.
public byte nextByte()	it scans the next token as a byte.
public short nextShort()	it scans the next token as a short value.
public int nextInt()	it scans the next token as an int value.
public long nextLong()	it scans the next token as a long value.
public float nextFloat()	it scans the next token as a float value.
public double nextDouble()	it scans the next token as a double value.

By: Shamama Anwar

38

User Input

2. Scanner class (Eg: 1)

```
import java.util.Scanner

class demo {
    public static void main(String args[])
    {
        int i;

        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number:");
        i = sc.nextInt();
        System.out.println("Number entered : " + i);
    }
}
```

3. DataInputStream / BufferedReader

By: Shamama Anwar

39

Java Operators and their precedence

- Operators
 - ✓Unary Operators
 - ✓Binary Operators
 - † Assignment Operators
 - † Arithmetic Operators
 - † Relational Operators
 - † Logical Operators
 - † Bitwise Operator
 - ✓Ternary Operator

By: Shamama Anwar

40

Comparing Entities

- A condition is an expression that can be true or false.
- Conditions in **if** statements can be formed by using the **equality operators** (== and !=) and **relational operators** (>, <, >= and <=).



- Program to compute average velocity by asking the user to enter the starting milepost, the ending milepost and the time elapsed in hours, minutes and seconds.
- Program to print the grade of a student by finding the average of 5 subject marks.

By: Shamama Anwar

31

Classes and Objects

- A **class** is a collection of **fields** (data) and **methods** (procedure or function) that operate on that data.
- The basic syntax for a class definition:

```
class ClassName [extends SuperClassName]
{ [fields declaration]
  [methods declaration]
}
```

- Method represents task in a program
 - Describes the mechanisms that actually perform its tasks
 - Hides from its user the complex tasks that it performs
 - Method call tells method to perform its task

By: Shamama Anwar

32

Methods

- A method is a construct for grouping statements together to perform a function.
- Enhances reusability of code. Can be defined by user or already defined in Java Library
- Defining a method (syntax)

```
modifier return_value_Type method_name(parameter_list)
{ //Method body }
```

- Example

```
public static int max (int a, int b)
{
    int c;
    if (a>b)
        c = a;
    else
        c = b;
    return c;
}
```

Method body

return value

By: Shamama Anwar

39

Methods

- Calling a method
- Example: Eg 1 (using a static function and no object created)

```
class demo
{
    public static int max (int a, int b)
    {
        int c;
        if (a>b) c = a;
        else c = b;
        return c;
    }

    public static void main(String args[])
    {
        int i = 5, j = 2;
        int k = max(i,j);
        System.out.println("The maximum number between
        "+i+" and "+j+" is: "+k);
    }
}
```

By: Shamama Anwar

40

Methods — Eg 2 (user interactive program, static function, No objects used)

```
import java.util.Scanner;
class demo
{
    public static int max (int a, int b)
    {
        int c;
        if(a>b)    c = a;
        else      c = b;
        return c;
    }
    public static void main(String args[])
    {
        int i, j, k;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number:");
        i = sc.nextInt();
        System.out.println("Enter a number:");
        j = sc.nextInt();
        k = max(i,j);
        System.out.println("The maximum number between "+i+"
and "+j+" is: "+k);
    }
}
```

By: Shamama Anwar

Methods— Eg 3 (user interactive program, objects used)

```
import java.util.Scanner;
class demo
{
    public int max (int a, int b)
    {
        int c;
        if(a>b)    c = a;
        else      c = b;
        return c;
    }
    public static void main(String args[])
    {
        int i, j, k;
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter a number:");
        i = sc.nextInt();
        System.out.println("Enter a number:");
        j = sc.nextInt();
        demo d = new demo();
        k = d.max(i,j);
        System.out.println("The maximum number between "+i+"
and "+j+" is: "+k);
    }
}
```

Programs

- WAP to design a simple four function calculator.
- WAP to reverse a number (with and without recursion).

By: Shamama Anwar

37

Methods

Method Overloading

- Methods of the same name can be declared in the same class, as long as they have different sets of parameters (determined by the number, types and order of the parameters) – this is called method overloading.
- When an overloaded method is called, the Java compiler selects the appropriate method by examining the number, types and order of the arguments in the call.
- Method overloading is commonly used to create several methods with the same name that perform the same or similar tasks, but on different types or different numbers of arguments.

By: Shamama Anwar

38

Methods

```
public class Overloading
{ int sq(int iVal)
  { System.out.println("Called square with argument:"+iVal);
    return (iVal*iVal); }
  double sq(double dVal)
  { System.out.println("Called square with argument:"+dVal);
    return dVal*dVal;}
}
public class OverloadingTest
{public static void main(String[] args)
  {Overloading f = new Overloading();
   System.out.println("Square of integer 8 is "+f.sq(8));
   System.out.println("Square of double 8.5 is"+f.sq(8.5));
  }
}
```

By: Shamama Anwar

39

Methods

Method Overloading

- Overloaded method calls cannot be distinguished by return type.
- Overloaded method declarations with identical signatures cause compilation errors, even if the return types are different.
- That is understandable, because the return type is not necessarily apparent when you call a method.

By: Shamama Anwar

40

Methods

```
public class Overloading
{
    int area(int iVal)
    { return (iVal*iVal); }

    double sq(int dVal)
    { return (3.14*dVal*dVal); }
}

public class OverloadingTest
{
    public static void main(String[] args)
    {
        Overloading f = new Overloading();
        System.out.println("Square Area = "+f.area(8));
        System.out.println("Circle Area = "+f.area(8));
    }
}
```

By: Shamama Anwar

41

Example

```
class Circle
{
    public double r;
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r; }
}
```

Creating Objects:

```
Circle acircle;
Circle bcircle;
```

No memory allocated to object. (Null reference)

Objects are created dynamically using the *new* keyword.

```
Circle acircle = new Circle();
Circle bcircle = new Circle();
```

By: Shamama Anwar

42

Example

```
class Circle
{
    public double r;
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r;}
}
public class demo
{ public static void main(String args[])
  { Circle aCircle = new Circle();
    aCircle.r = 2.45;
    System.out.println("Area = "+aCircle.area()+"
    "Circumference = "+aCircle.circumference());
  }
}
```

By: Shamama Anwar

43

Example

```
import java.util.Scanner;
class Circle
{
    public double r;
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r;}
}
public class demo
{ public static void main(String args[])
  { Circle aCircle = new Circle();
    Scanner sc = new Scanner(System.in);
    System.out.println("Enter the radius:");
    aCircle.r = sc.nextDouble();
    System.out.println("Area = "+aCircle.area()+"
    "Circumference = "+aCircle.circumference());
  }
}
```

By: Shamama Anwar

44

Example

```
aCircle = new Circle();
bCircle = new Circle() ;
.....
bCircle = aCircle;
```

By: Shamama Anwar

45

Modifiers

Modifiers that can be used while declaring methods

Modifier	Description
public	Can be accessed by methods of the same class, and methods of other classes.
protected	Can be accessed by methods of the same class, methods of subclasses, and methods of classes in the same package.
private	Can be accessed by methods of the same class only.
default	Can be accessed by methods in the same package only.
static	The method can be invoked without creating an object of the class
abstract	The class must be extended and the abstract method must be overridden in the subclass
final	The method cannot be overridden in a subclass.
synchronized	Used in multithreading
throws	A list of exceptions is thrown from this method

By: Shamama Anwar

46

Example

```
class Circle
{
    private double r;
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r;}
}
public class demo
{
    public static void main(String args[])
    {
        Circle aCircle = new Circle();
        aCircle.r = 2.45;
        System.out.println("Area = "+aCircle.area()+"
        "Circumference = "+aCircle.circumference())
    }
}
```

ERROR

By: Shamama Anwar

47

Example

```
import java.util.Scanner;
class Circle
{
    private double r;
    public double circumference()
    { return 2*3.14*r; }
    public double area()
    { return 3.14 * r * r;}
}
public class demo
{
    public static void main(String args[])
    {
        Circle aCircle = new Circle();
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the radius:");
        aCircle.r = sc.nextDouble();
        System.out.println("Area = "+aCircle.area()+"
        "Circumference = "+aCircle.circumference());
    }
}
```

ERROR

By: Shamama Anwar

48

Example

```
class Circle
{
    private double r;
    void getdata()
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter the radius:");
        r = sc.nextDouble();
    }
    public double circumference()
    {
        return 2*3.14*r;
    }
    public double area()
    {
        return 3.14 * r * r;
    }
}

public class demo
{
    public static void main(String args[])
    {
        Circle aCircle = new Circle();
        aCircle.getdata();
        System.out.println("Area"+aCircle.area()+
            "Circumference"+aCircle.circumference());
    }
}
```

By: Shamama Anwar

49

Wrapper Classes

- The `java.lang` package contains *wrapper classes* that correspond to each primitive type:

Primitive Type	Wrapper Class	Primitive Type	Wrapper Class
boolean	Boolean	float	Float
byte	Byte	int	Integer
char	Character	long	Long
double	Double	short	Short

- Wrapper classes also contain a number of useful predefined constants and static methods

By: Shamama Anwar

50

Wrapper Classes and some methods

- To create objects from these wrapper classes a value is passed to the constructor.

```
Integer number = new Integer(7);
```

- Each of the numeric wrapper classes has a static method that converts a string to a number.
 - The **Integer** class has a method that converts a String to an int,
 - The **Double** class has a method that converts a String to a double,
 - And so on.....
- These methods are known as *parse methods* because their names begin with the word "parse."

By: Shamama Anwar

31

Wrapper Classes – parse methods

```
byte bVar = Byte.parseByte("1");  
int iVar = Integer.parseInt("2599");  
short sVar = Short.parseShort("10");  
long lVar = Long.parseLong("15908");  
float fVar = Float.parseFloat("12.3");  
double dVar = Double.parseDouble("7945.6");
```

The parse methods all throw a `NumberFormatException` if the String object does not represent a numeric value.

By: Shamama Anwar

32

Wrapper Classes – valueOf method

- Each of the numeric wrapper classes has a static valueOf method that returns the wrapper object corresponding to the primitive value.
- This function is overloaded: one version accepts numeric values and the other accepts String.

```
int i = 12;  
Double vald = Double.valueOf("98.95");  
Integer b = Integer.valueOf(i);
```

By: Shamama Anwar

33

Wrapper Classes – toString method

- Each of the numeric wrapper classes has a static toString method that converts a number to a String.
- The method accepts the number as its argument and returns a String representation of that number.

```
int i = 12;  
double d = 14.95;  
String str1 = Integer.toString(i);  
String str2 = Double.toString(d);
```

By: Shamama Anwar

34

Wrapper Classes – other methods

- Each of the numeric wrapper classes have a non- static method which can be used to convert a numeric wrapper class to their respective primitive numeric types.

Methods	Purpose
byteValue()	returns the value of this Integer as a byte
doubleValue()	returns the value of this Integer as an double
floatValue()	returns the value of this Integer as a float
intValue()	returns the value of this Integer as an int
shortValue()	returns the value of this Integer as a short
longValue()	returns the value of this Integer as a long

```
Integer ob = new Integer(5);
int x = ob.intValue();
```

By: Shamama Anwar

35

Wrapper Classes – other methods

The Integer and Long classes have three additional methods:
toBinaryString, toHexString, and toOctalString

```
int number = 14;
System.out.println(Integer.toBinaryString(number));
System.out.println(Integer.toHexString(number));
System.out.println(Integer.toOctalString(number));
```

The numeric wrapper classes each have a set of static final variables

MIN_VALUE and
MAX_VALUE.

These variables hold the minimum and maximum values for a particular data type.

```
System.out.println("The minimum value for int is "
+ Integer.MIN_VALUE);
```

```
System.out.println("The maximum value for int is
+ Integer.MAX_VALUE);
```

By: Shamama Anwar

36

Autoboxing and Unboxing

- The process of going from a value of a primitive type to an object of its wrapper class is called autoboxing or boxing.
- To convert a primitive value to an "equivalent" class type value, create an object of the corresponding wrapper class using the primitive value as an argument
- The new object will contain an instance variable that stores a copy of the primitive value

```
Integer integerObject = new Integer(42);
```

By: Shamama Anwar

37

Autoboxing and Unboxing

- The process of going from an object of a wrapper class to the corresponding value of a primitive type is called unboxing.
- The methods for converting an object from the wrapper classes **Byte**, **Short**, **Integer**, **Long**, **Float**, **Double**, and **Character** to their corresponding primitive type are (in order) **byteValue**, **shortValue**, **intValue**, **longValue**, **floatValue**, **doubleValue**, and **charValue**
- None of these methods take an argument

```
Integer integerObject = new Integer(42);
int i = integerObject.intValue();
```

By: Shamama Anwar

38

Example – Primitive to Wrapper (Boxing)

```
public class WrapperExample1
{
    public static void main(String args[])
    {
        int a=20;
        Integer i=Integer.valueOf(a);
        Integer j=a;
        System.out.println(a+" "+i+" "+j);
    }
}
```

By: Shamama Anwar

29

Example – Wrapper to Primitive (Unboxing)

```
public class WrapperExample2
{
    public static void main(String args[])
    {
        Integer a=new Integer(3);
        int i=a.intValue();
        int j=a;
        System.out.println(a+" "+i+" "+j);
    }
}
```

By: Shamama Anwar

30

Wrapper Classes – Comparison methods

int compareTo(int i) - Compares the numerical value of the invoking object with that of i. Returns 0 if the values are equal. Returns a negative value if the invoking object has a lower value. Returns a positive value if the invoking object has a greater value.

static int compare(int num1, int num2) - Compares the values of num1 and num2. Returns 0 if the values are equal. Returns a negative value if num1 is less than num2. Returns a positive value if num1 is greater than num2.

boolean equals(Object intObj) - Returns true if the invoking Integer object is equivalent to intObj. Otherwise, it returns false.

By: Shamama Anwar

61

Wrapper Classes – Comparison methods

```
public class WrapperDemo
{
    public static void main (String args[])
    {
        Integer intObj1 = new Integer (25);
        Integer intObj2 = new Integer ("25");
        Integer intObj3= new Integer (35);
        //compareTo demo
        System.out.println("Comparing using compareTo Obj1 and Obj2: " +
            intObj1.compareTo(intObj2));
        System.out.println("Comparing using compareTo Obj1 and Obj3: " +
            intObj1.compareTo(intObj3));
        //Equals demo
        System.out.println("Comparing using equals Obj1 and Obj2: " +
            intObj1.equals(intObj2));
        System.out.println("Comparing using equals Obj1 and Obj3: " +
            intObj1.equals(intObj3));
    }
}
```

By: Shamama Anwar

62

Wrapper Classes – Comparison methods

```
Float f1 = new Float("2.25f");
Float f2 = new Float("20.43f");
Float f3 = new Float(2.25f);
System.out.println("Comparing using compare f1 and f2: " +
Float.compare(f1,f2));
System.out.println("Comparing using compare f1 and f3: " +
Float.compare(f1,f3));
//Addition of Integer with Float
Float f = intObj1.floatValue() + f1;
System.out.println("Addition of intObj1 and f1: "+ intObj1 +"+" +
f1+"=" +f );
}
}
```

By: Shamama Anwar

63

Character Class

- Character object can be created using the Character class constructor:
Character obj = new Character('a');
- If a primitive char is passed into a method that expects an object, the compiler automatically converts the char to a Character (autoboxing)
- Some methods in Character are:
 - **isLetter()** – Determines whether the specified char value is a letter
 - **isDigit()** – Determines whether the specified char value is a digit
 - **isWhiteSpace()** – Determines whether the specified char value is white space.
 - **isUpperCase()** – Determines whether the specified char value is uppercase.
 - **isLowerCase()** – Determines whether the specified char value is lowercase.
 - **toUpperCase()** – Returns the uppercase form of the specified char value.
 - **toLowerCase()** – Returns the lowercase form of the specified char value.
 - **toString()** – Returns a String object representing the specified character value that is, a one-character string.

By: Shamama Anwar

64

Character Class

```
public class charToCharacter
{ public static void main(String[] args)
{
    char c = 'a';
    Character cObj = new Character(c);
    System.out.println(cObj);
}
}
```

```
public class CharTest
{ public static void main(String args[])
{
    boolean b1 = Character.isDigit('5');
    boolean b2 = Character.isDigit('-');
    System.out.println(b1);
    System.out.println(b2);
}
}
```

By: Shamama Anwar

65

Character Class

```
public class CharTest
{ public static void main(String args[])
{
    boolean b1 = Character.isLetter('S');
    boolean b2 = Character.isLetter('r');
    boolean b3 = Character.isLetter('#');
    System.out.println(b1);
    System.out.println(b2);
    System.out.println(b3);
}
}
```

By: Shamama Anwar

66

Character Class

```
public class CharTest
{ public static void main(String args[])
{   boolean b1 = Character.isWhiteSpace(' ');
    boolean b2 = Character.isWhiteSpace('\t');
    boolean b3 = Character.isWhiteSpace('\n');
    boolean b4 = Character.isWhiteSpace('3');
    System.out.println(b1);
    System.out.println(b2);
    System.out.println(b3);
    System.out.println(b4);
}
}
```

By: Shamama Anwar

67

Character Class

```
public class CharTest
{ public static void main(String args[])
{   boolean b1 = Character.isUpperCase('R');
    boolean b2 = Character.isUpperCase('n');
    boolean b3 = Character.isLowerCase('R');
    boolean b4 = Character.isLowerCase('n');
    System.out.println(b1);
    System.out.println(b2);
    System.out.println(b3);
    System.out.println(b4);
}
}
```

By: Shamama Anwar

68

Character Class

```
public class CharTest
{ public static void main(String args[])
{   char c1 = Character.toUpperCase('r');
    char c2 = Character.toUpperCase('n');
    char c3 = Character.toLowerCase('E');
    char c4 = Character.toLowerCase('L');
    System.out.println(c1);
    System.out.println(c2);
    System.out.println(c3);
    System.out.println(c4);
}
}
```

```
public class CharTest
{ public static void main(String args[])
{   String s1 = Character.toString('r');
    System.out.println(s1);
}
}
```

69

String Class

- An object of the String class represents a string of characters.
- The String class belongs to the `java.lang` package, which does not require an import statement.
- Like other classes, String has constructors and methods.
- Unlike other classes, String has two operators, + and += (used for concatenation).

```
String word = new String("Java");
String word2 = new String(word);
```

```
String word = "Java";
String word2 = word;
```

```
char[] letters = {'J', 'a', 'v', 'a'};
String word = new String(letters); // "Java"
```

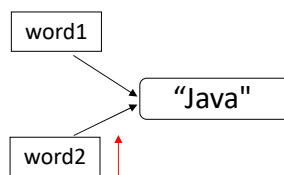
By: Shamama Anwar

70

String Class

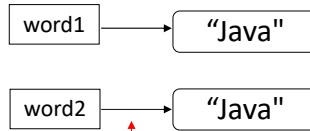
- Once created, a string cannot be changed: none of its methods changes the string. Such objects are called *immutable*.
- Immutable objects are convenient because several references can point to the same object safely: there is no danger of changing an object through one reference without the others being aware of the change.
- Advantages of Immutability: Uses less memory.

```
String word1 = "Java";
String word2 = word1;
```



OK

```
String word1 = "Java";
String word2 = new String(word1);
```

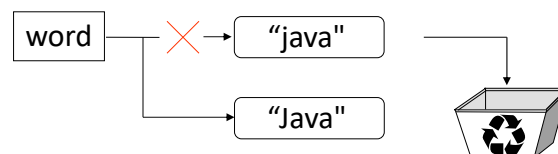
Less efficient:
wastes memory

71

String Class

- Disadvantages of Immutability: Less efficient (a new string needs to be created even for a small change)

```
String word = "Java";
char ch = Character.toUpperCase(word.charAt(0));
word = ch + word.substring(1);
```



- Empty String: An empty String has no characters. It's length is 0.

```
String word1 = "";
String word2 = new String();
```

By: Shamama Anwar

72

String Class

- Some methods in Character are:
 - length()** - returns the number of characters in the string
 - toUpperCase()** - converts the String to Upper case
 - toLowerCase()** - converts the String to lower case

```
String s = "Window";
System.out.println(s.toUpperCase());
System.out.println(s.toLowerCase());
System.out.println(s);
```

- charAt(int)** - returns the character at position specified by integer.

```
String s = "Window";
System.out.println(s.charAt(2));
```

- trim()** - eliminates white spaces before and after string

```
String s = " Window ";
System.out.println(s.trim());
```

By: Shamama Anwar

73

String Class

- startsWith(String)** - returns true if the calling object starts with the string specified in the argument
- endsWith(String)** - returns true if the calling String object ends with the string specified in the argument.

```
String s = "Window";
System.out.println(s.startsWith("Win"));
System.out.println(s.endsWith("f"));
```

- valueOf()** - converts primitive datatype into String.

```
int a=10;
String s=String.valueOf(a);
System.out.println(s+10);
```

- substring(int i)** - returns a the substring from the i^{th} character to the end.
- substring(int i, int k)** - returns the substring from the i^{th} character to the $(k-1)^{\text{th}}$ character.

```
String s = "Windows";
System.out.println(s.substring(2));
System.out.println(s.substring(2,5));
```

74

String Class

- **concat(String)** – concatenates the string in the invoking object with the string in the parameter

```
String s = "Window";
String s1 = "OS"
System.out.println(s.concat(s1));
System.out.println(s+s1);
```

- **replace(char old, char new)** - replaces all occurrences of character old with character new in the invoking object.

```
String s = "year"
System.out.println(s.replace('y','r'));
```

By: Shamama Anwar

75

StringBuffer Class

- **java.lang.StringBuffer** provides a String object that can be amended.
- The StringBuffer class is provided for strings that may need to be changed.

```
String s = "Window";
```

here the string "Windows" cannot be changed.

- To overcome this limitation the StringBuffer class is used that accommodates strings that are not immutable.
- It uses a buffer that is much larger than any one string; actual size of the buffer is the capacity
- Provides methods that can change individual characters within a string.

```
StringBuffer s = new StringBuffer("Window");
StringBuffer s1 = "Windows";
```

By: Shamama Anwar

76

StringBuffer Class

- Has four constructors:
 - **public StringBuffer()** constructs a StringBuffer with no characters and a default size of 16 characters
 - **public StringBuffer(int Capacity)** creates a StringBuffer with no characters and a capacity defined by the parameter
 - **public StringBuffer(String s)** contains the same characters as those stored in the String object s
- An object created as a StringBuffer can easily be converted to an object of the String class if needed using the **toString()** method.

By: Shamama Anwar

77

StringBuffer Class

- Some methods in the StringBuffer class are:
 - **length()** - returns the length of the string (character count)

```
public class StringBufferDemo
{
    public static void main(String[] args)
    {
        StringBuffer buff = new StringBuffer("Tutorials");
        System.out.println("length = " + buff.length());
        buff = new StringBuffer("");
        System.out.println("length = " + buff.length());
    }
}
```

By: Shamama Anwar

78

StringBuffer Class

- **setlength(int newLength)** - sets the length of the string. If the **newLength** argument is greater than or equal to the current length, sufficient null characters ('\u0000') are appended so that length becomes the **newLength** argument.

```
public class StringBufferDemo
{
    public static void main(String[] args)
    {
        StringBuffer buff = new StringBuffer("tutorials");
        System.out.println("buffer1 = " + buff);
        System.out.println("length = " + buff.length());
        buff.setLength(5);
        System.out.println("buffer2 = " + buff);
        System.out.println("length = " + buff.length());
    }
}
```

By: Shamama Anwar

79

StringBuffer Class

- **capacity()** -method returns the current capacity. The capacity is the amount of storage available for newly inserted characters.

```
public class StringBufferDemo
{
    public static void main(String[] args)
    {
        StringBuffer buff = new StringBuffer("TutorialsPoint");
        System.out.println("capacity = " + buff.capacity());
        buff = new StringBuffer(" ");
        System.out.println("capacity = " + buff.capacity());
    }
}
```

By: Shamama Anwar

80

StringBuffer Class

- **append(String str)** - appends the specified string to the character sequence in the object. The characters of the String argument are appended, in order, increasing the length of this sequence by the length of the argument.

```
public class StringBufferDemo
{ public static void main(String[] args)
  { StringBuffer buff = new StringBuffer("tutorials ");
    System.out.println("buffer = " + buff);
    buff.append("point");
    System.out.println("After append = " + buff);
    buff = new StringBuffer("1234 ");
    System.out.println("buffer = " + buff);
    buff.append("!#$%");
    System.out.println("After append = " + buff);
  }
}
```

By: Shamama Anwar

81

StringBuffer Class

```
public StringBuffer append(boolean b)
public StringBuffer append(char c)
public StringBuffer append(char[] str)
public StringBuffer append(char[] str,int offset,int len)
public StringBuffer append(double d)
public StringBuffer append(float f)
public StringBuffer append(int i)
public StringBuffer append(long l)
public StringBuffer append(Object obj)
public StringBuffer append(StringBuffer sb)
public StringBuffer append(String str)
```

By: Shamama Anwar

82